

The Architect's Role in Community Shepherd

Damian A. Tamburri, Politecnico di Milano

Rick Kazman, University of Hawaii

Hamed Fahimi, CGI

// Community shepherds guide development projects' social and organizational structures, look for early warning signs of trouble, and help mitigate that trouble. Knowing community smells—ways in which software communities might be “buggy”—can help with this. //



SOFTWARE ARCHITECTS DON'T just design architecture components or champion architecture qualities; they often must guide and harmonize the entire community of project stakeholders. The community-shepherding aspects of the architect's role have been gaining attention, given the increasing importance of complex “organizational rewiring” scenarios such

as DevOps, open source strategies, transitions to agile development, and corporate acquisitions.

To help architects in their role as community shepherds, we present guidance on how to tell when a community has problems and what to do about those problems. We also provide an overview of community types.

Interior Design for Software Architects

What do you do when you want to reorganize the furniture in your room? You take pencil and paper and draw a blueprint of the room and its elements (the desk and wardrobes, that “exotic” table your partner got who-knows-where, and so on), including the most important properties (size, shape, fragility, and so on). Now let's say your partner wants to keep that exotic table where it is without even removing the dreadful tapestry on top of it. What do you do? You get some counseling.

Well, the same story happens in software architecture practice.¹ Architects often must act as key community figures or counselors who bridge the gap between the development organization, including its subgroups, and the software architecture, including its properties and implementation details. Consequently, those software architectures (the blueprints in this case) are often augmented with views that express authority,² task divisions, elements, relations, and more. These characteristics reflect organizational and community types known to organizations and social-network researchers.^{3,4} (A community type is a social network in which social or organizational characteristics are constantly evident.³ For example, a project team is small [size], cross-functional [average cognitive distance], and time-bound [duration].)

Returning to our room: Counseling isn't going well, but you love your partner and decide to give in on the table—but you never forget. If this happens regularly, tension or even defiance might ensue.

Well, the same story happens in software architecture practice.⁵ Architects have often reported trouble



while participating in “smelly” (sub-optimal, either organizationally or technically) communities. They’ve had to take on the role of intermediaries of the entire community. This has involved proposing mitigations for good-then-bad implementations that incur technical debt⁶ or good-then-bad organizational and social decisions (for example, releasing a critical component as open source, which then incurs social debt⁵).

Returning to our story: Imagine you want to change not just the room but the type of house or even its location. The simple room blueprint won’t be enough, but it’s all you have as a starting point for planning the move.

Well, the same story happens with software architecture practitioners.² Software architects often must play a key role in organizational- or technical-change scenarios—for example, organizational mergers, outsourcing, process changes (such as shifting to Scrum or DevOps), or adopting a new technology stack. Such scenarios require that architects

- adapt both the organization and architecture to accommodate the changes;
- continuously change the organization and architecture to keep the two aligned; and
- understand and mediate existing and new requirements (for example, continuous integration), stakeholders (for example, infrastructure engineers), and concerns (for example, integration speed and test coverage).

In these scenarios, architects must strike a balance between community and architecture—for example, by adapting their architecture to support continuous improvement

while molding a community that welcomes a DevOps strategy. In these circumstances, the architect’s role is critical in balancing proposed technical solutions against challenging social and organizational issues: *community smells*^{7,8}—ways in which software communities might be “buggy.”

So, how do architects cope with their (largely implicit) role as community managers, counselors, and shepherds? Little literature or curriculum exists to prepare them for the strain of this role. Shepherding involves creating effective synergies

organizations of varying sizes. One organization was one of the world’s largest IT services firms, with more than 65,000 employees. Another was a medium-size software engineering consulting firm from Italy with more than 50 employees. Others were agile small-to-medium enterprises with dozens of employees who combined software architecting with community contributions.

This study had three goals. First, we wanted to understand how software architects can know if their community smells. Toward that end, we discovered 12 community smells.

We wanted to understand how software architects can know if their community smells.

between people such that development community goals are aligned with good-quality software architectures, their elements, and their properties. There are no proven techniques architects can employ to deal with their job’s social and organizational aspects. Most of their efficacy in organizational-change scenarios depends on their personal experience. Furthermore, this efficacy is even more critical with the IT industry’s increasing interest in the organizational-rewiring scenarios we mentioned before.

Our Study

To ease architects’ lives and provide them strategies to cope with their role of community shepherd, we conducted an industrial study employing interviews, focus groups, and workshops with 51 software engineering professionals from four

These smells are recurrent—each was reported 10 times or more in our study, and architects will likely have to cope with them.

Second, we wanted to identify what software architects should *do* if their community smells. So, we elicited and documented possible strategies to address the smells.

Finally, we wanted to identify common community types and their features because these provide patterns for structuring a software engineering community. We indicated each feature’s importance to each community type³ because this helps architects determine how best to shepherd those communities.¹

Know Your Community Smells

Much like code smells, community smells are initially hunches that something might be going wrong.

TABLE 1

Community smells and what software architects should know about them.

Smell*	Description	Context	Cause	Consequences
Time Warp (27)	A change in organizational structure and process that leads people to wrongly assume that communication will take less time and that explicit additional coordination isn't needed	• Formal networks • Working groups • Communities of practice • Formal groups	Experience diversity	• Low software architecture quality • Malfunctioning software or code smells • Losing face in the community • Unsolved operations issues • Unsatisfied customers
Cognitive Distance (24)	The distance developers perceive on the physical, technical, social, and cultural levels regarding peers with considerable background differences ¹⁰	• Formal networks • Working groups • Communities of practice • Formal groups • Project teams	Experience diversity	• Wasted time • Pitting newbies versus experts • Faulty or smelly code • Additional development costs • Wasted operations resources • Lack of an optimal understanding across different operations areas • Mistrust across the development network • Misinterpretation of expectations
Newbie Free-Riding (24)	Newcomers being left to themselves regarding understanding what to do and for whom, with the consequent free-riding of older employees (that is, the economic free-rider problem applied to software engineering)	• Formal groups • Project teams	—	• High work pressure • Irritation • Demotivation of non-senior members
Power Distance (24)	The distance that less powerful or less responsible members of a software development community perceive, accept, or expect with power holders	• Formal networks • Communities of practice	Lack of architecture knowledge sharing	• Additional project costs • Financial loss • Lost bids
Disengagement (24)	Thinking the product is mature enough and sending it to operations even though it might not be ready	• Communities of practice • Formal networks	• Lack of engagement in development • Lack of curiosity	• Missing software development contextual information • Wild assumptions
Priggish Members (13)	Demanding of others (for example, operations) pointlessly precise conformity or exaggerated propriety, especially in a self-righteous or irritating manner	Communities of practice	—	• Additional project costs • Frustrated team members

The software development community “works,” but there’s smoke on the horizon, warning of problems arising in the near future. The architect is increasingly the one who uncovers community smells and associated social debt, just as he or she

deals with code smells and technical (and architectural) debt.⁹

Table 1 showcases the community smells we found in our study, focusing on

- previously unreported smells and

- smells directly related to architectures and the role of architects.

For example, Table 1’s first row describes the Time Warp smell, which involves wrong assumptions

TABLE 1 CONT.

Community smells and what software architects should know about them.

Smell*	Description	Context	Cause	Consequences
Cookbook Development (13)	Developers who are stuck in their ways and who refuse innovative ideas or ways of working (for example, agile methods or DevOps)	Communities of practice	Thinking in an old framework—for example, the waterfall model	• Mismatched expectations between the customer and the rest of the community
Institutional Isomorphism (11)	The similarity of the processes or structure of one subcommunity to those of another, whether the result of imitation or independent development under similar constraints	• Formal networks • Communities of practice • Formal groups	• Excessive conformity to standards • Lack of innovation • Using a formal structure to achieve community goals • Rigid thinking from different parts of the community	• A negative impact on team spirit • A less flexible or static product • Lack of innovation • Stagnation • Lack of communication or collaboration
Hyper-Community (14)	A hyperconnected community that's sensible of groupthink but also influences its subcommunities	• Communities of practice • Formal networks	—	• Increased turbulence • Buggy software
DevOps Clash (29)	Clashes in the mix between development and operations from multiple geographical locations, with contractual obligations to either development or operations	• Formal networks • Formal groups	Geographic dispersion	• Increased project costs • Lack of trust-building • The inability to bridge between different thought worlds across development and operations • “Stickiness” of knowledge transfer • Clashes between the development and operations cultures • Slower development • Ineffective operations
Informality Excess (10)	Excessive informality due to the relative absence of information management and control protocols	Project teams	—	• Low accountability of both development and operations staff • Information spillover
Unlearning (12)	A new technological or organizational advancement or best practice (for example, as part of training courses) that becomes infeasible when shared with older members	• Project teams • Formal groups	Experience diversity	• Lack of engagement • Gradual loss of the new knowledge or best practice

* The numbers in parentheses indicate the number of times that smell appeared in our study.

regarding the development and operations time frames after an organizational change—for example, outsourcing architecture elements. According to our data, Time Warp

likely occurs in four community types: formal networks, working groups, communities of practice, and formal groups. These four types share key features that might require

additional attention during organizational changes; we discuss these features in more detail later.

Moreover, we found that Time Warp correlates with high experience

diversity across community members; this diversity might require the architect's attention. This was the case in seven instances of Time Warp (approximately 26 percent of all occurrences of the smell). Finally, Time Warp leads to a hastily developed architecture, resulting in poor design, and hastily developed code, resulting in code smells.

To give you an additional whiff of community smells and their relation to software architecture, let's return to our story. You're moving to a new town, and you hire a decorator to arrange your furniture. The Cognitive Distance smell represents the distance (in assumptions, attitudes, emotions, and knowledge) you perceive between yourself and your decorator (who finds your furniture distasteful). In a software development community, Cognitive Distance refers to the differences in technical knowledge and skills—and hence norms and assumptions—among developers.

The Newbie Free-Riding smell refers to when your decorator is a novice and hasn't received sufficient details about your furniture or decoration tastes. Similarly, during organizational rewiring, newcomers might suffer from a lack of awareness of project or architecture norms.¹¹ So, additional time and effort must be invested in disseminating such knowledge.

Moreover, remember when you were a young developer looking up to veteran colleagues, who made all the decisions without consulting you (or, at times, without even communicating them to you)? Well, the same story happens in software architecture practice. Researchers have studied how architects' loneliness can disrupt software processes¹¹ and decrease architecture

quality.⁵ This is the effect of the Power Distance smell.

Architects can combat Power Distance's negative consequences by reaching out to their communities—for example, by building architecture boards.⁷ Or, architects can rearrange architecture elements and their responsibilities to redistribute authority more homogeneously—for example, by increasing modularity and allocating new, smaller modules to smaller teams. Note here the architect's crucial role in designing and shepherding the community, the architecture, or both to achieve better alignment.

These community smells provide at least two benefits. First, they're starting points for you to study. They help refine your understanding of the architect's role in community shepherding. Second, they're hints for caution that should be added to a software architect's handbook.

Architects ... React!

Table 2 presents the mitigations the architects in our study used to successfully tackle the community smells in Table 1.

To give you a taste of these mitigations, we illustrate several examples from our study. These mitigations can help software architects tackle potentially hazardous circumstances arising around community smells. Furthermore, architects can proactively adopt these approaches to avoid the conditions in which smells emerge.

Buddy Pairing

Our evidence suggests that modularizing architecture elements such that they can be arranged over multiple remote sites should engender the creation of *boundary*

spanners—people whose architecture knowledge spans not only development (or operations) sites but also components.⁷ For example, in our study, the architecture in three projects was modularized into (at most) two essential components per site. People were then bound in pairs across components and sites. These work buddies regularly updated each other on progress, incidents, or similar architecture knowledge. This architecture-based buddy-pairing strategy aimed to mitigate Cognitive Distance, with positive results in all three cases, irrespective of the software process.

Sociotechnical-Risks Engineering

Also, our evidence suggests that software architecture can be designed defensively from a sociotechnical perspective. Consequently, critical architecture elements remain tightly controlled by an organization and loosely coupled with respect to “outsiders.” That organization can then dedicate resources to mitigating architecture risks associated with these critical elements. This is particularly appropriate for architecting for development in networked organizations.²

Some of our study participants adopted this architecture-based sociotechnical-risk-engineering strategy to mitigate Time Warp. For example, seven interviewees in three organizations reported on four projects in which a (once colocated) software project became distributed and their organization changed to a (more expensive) bus-based architecture. The aim was to retain control over the bus while partner sites developed extensions on top of the bus. This eliminated Time Warp, which had been reported in dozens of prior projects that hadn't adopted this strategy.

Smells and mitigations.

Smell	Mitigation
Time Warp	Better syncing of coordination and communication of architecture decisions, and better time estimation or evidence-based scheduling
	Architect-as-a-coach
	Devoting more resources to risk engineering
Cognitive Distance	Professional communication intermediaries
	Designing an architecture on the basis of the perceived cognitive distance
	Stimulating engagement in newcomers, and designing architecture to accommodate buddy pairing
	Architecture knowledge exchange through redocumentation, workshops, and presentations
	Architect-as-a-coach
	Cross-functional and community-wide review of code, designs, and operations procedures
Newbie Free-Riding	Coordination management and engagement creation
	Architect-as-a-coach
	Explicit empowerment of architecture decision changes
	Organizational monitoring—for example, anonymous mood polling
Power Distance	—
Disengagement	A DevOps shift-left approach to address issues earlier during development, and having operations staff also be developers
	Designing a people-oriented rather than feature-oriented architecture
Priggish Members	Harmonizing responsibilities, and using Scrum and agile methods
Cookbook Development	Using agile methods—in particular, employing the protecting and interface role of Scrum master, who communicates the customer organization's demands to the development and operations teams
	Expectation management, and managers and architects using knowledge brokers to disseminate and oversee expectations
Institutional Isomorphism	Open communication and informality, and model-based mediation of knowledge
	Daily stand-ups, and keeping track of actions, agreements, and expectations through work-division artifacts
Hyper-Community	Architect-as-a-coach
	Inciting doubt through discussion and reverse logic
DevOps Clash	Community-wide engagement, including operations staff, clients, and user panels
	Nearshoring (For example, Poland is closer to the Netherlands than China is.)
	Standardization of software design and implementation, then separation between development and operations, and then operations offshoring
	Knowledge brokers, and a new service or community coordinator for efficient working
Informality Excess	—
Unlearning	Ad hoc individual training for the more experienced developers
	A more active organizational approach and resources for architecture knowledge sharing

TABLE 3

Community types and their features.*

Feature	Community type												
	Community of practice	Informal network	Formal network	Informal community	Network of practice	Working group	Project team	Formal group	Problem-solving community	Learning community	Social network site	Knowledge community	Strategic community
Community structure	High	High	High	High	High	High	High	High	High	High	High	High	High
Situatedness	High	Low	Low	Low	Low	High	High	High	Low	Low	Low	Low	Low
Dispersion	Low	High	High	High	High	Low	Low	Low	High	Low	Low	High	High
Problem solving	Low	Low	Low	Low	Low	High	High	High	High	Low	Low	Low	High
Informality	High	High	Low	Low	High	High	High	High	High	High	Low	High	Low
Formality	Low	Low	High	Low	Low	High	High	High	Low	Low	Low	High	High
Engagement	High	High	Low	High	Low	High	High	Low	High	High	Low	Low	Low
Cohesion	Low	Low	High	Low	Low	High	High	Low	Low	Low	Low	Low	Low
Duration	High	High	High	High	High	High	Low	High	High	High	Low	Low	High
ROI tracking	Low	Low	Low	Low	Low	Low	High	Low	Low	Low	Low	Low	High
Governance	Low	Low	High	Low	Low	High	High	High	Low	Low	Low	Low	High
Culture	Low	Low	Low	Low	Low	High	Low	Low	High	High	Low	Low	Low
Visibility	Low	Low	Low	Low	Low	Low	Low	Low	Low	Low	Low	High	Low

* Low means the feature is unimportant or irrelevant; high means the feature is salient and primary. ROI means return on investment.

Mood Polling

Moreover, our evidence suggests that constantly monitoring progress and organizational procedures on the basis of architecture elements can reduce the proliferation of socio-technical problems or community smells.^{5,7} Any problems or smells will be reported as they emerge.

For example, some of our study participants used architecture-based anonymous mood polling to mitigate Newbie Free-Riding. This provided two benefits:

- Everyone could share their mood regarding the organizational

scenario and its mapping onto architecture elements.

- Architects could make further design decisions for improved organizational coordination—for example, by increasing modularization and empowering developers to deal with “bad mood” architecture elements.

This mitigation revealed both community and technical smells more than 20 times across several projects. Remarkably, we’ve seen start-ups emerge that aim to mitigate Newbie Free-Riding; examples include Celkee

(www.celkee.com) and HappyOrNot (www.happy-or-not.com/en).

Shift Left

Finally, our evidence suggests it’s not enough to address organizational and operational concerns (for example, infrastructure configuration) sooner, nor is it enough to use architectures just for division of work. The architecture must take into account key organizational concerns (discussed in the next section) whose counter-effects would normally become apparent during delivery (such as handover procedures or developer-to-operator coupling).

For example, five projects in our study employed an architecture-based DevOps shift-left strategy.¹² Such a strategy shifts organizational and operational concerns (for example, development-to-operations team mixing) to earlier (“left”) in the life cycle toward architecting and development. This approach effectively mitigated Disengagement, which was decreasing developer and operations staff engagement and limiting their interactions over joint operational procedures. With this strategy, the organizational structures were re-designed, creating a small DevOps team (including both development and operations people) for each architecture element.

Community Types Explained

Given that the community types and their features might “bug” architectures and the architects’ efforts to communicate and maintain them, Table 3 lists 13 key community types investigated in social-network and organizational research, focusing on their features for comparison. The table indicates the features’ importance to the community types. Notably, many of these types are the ones in which we observed community smells.

These community types and their features provide patterns for structuring (and shepherding) a software engineering community. For example, formality might be a primary characteristic in scenarios with well-defined rules and regulations, typically dictated by corporate governance. Conversely, informality or engagement might occur primarily in communities relying on informal communication—for example, open source communities. Moreover, cohesion and duration are primarily in

PURPOSE: The IEEE Computer Society is the world’s largest association of computing professionals and is the leading provider of technical information in the field.

MEMBERSHIP: Members receive the monthly magazine *Computer*, discounts, and opportunities to serve (all activities are led by volunteer members). Membership is open to all IEEE members, affiliate society members, and others interested in the computer field.

OMBUDSMAN: Email ombudsman@computer.org.

COMPUTER SOCIETY WEBSITE: www.computer.org

Next Board Meeting: 13–14 November 2016, New Brunswick, NJ, USA

EXECUTIVE COMMITTEE

President: Roger U. Fujii

President-Elect: Jean-Luc Gaudiot; **Past President:** Thomas M. Conte; **Secretary:** Gregory T. Byrd; **Treasurer:** Forrest Shull; **VP Member & Geographic Activities:** Nita K. Patel; **VP, Publications:** David S. Ebert; **VP, Professional & Educational Activities:** Andy T. Chen; **VP, Standards Activities:** Mark Paulk; **VP, Technical & Conference Activities:** Hausi A. Müller; **2016 IEEE Director & Delegate Division VIII:** John W. Walz; **2016 IEEE Director & Delegate Division V:** Harold Javid; **2017 IEEE Director-Elect & Delegate Division V:** Dejan S. Milošević

BOARD OF GOVERNORS

Term Expiring 2016: David A. Bader, Pierre Bourque, Dennis J. Frailey, Jill I. Gostin, Atsuhiko Goto, Rob Reilly, Christina M. Schober

Term Expiring 2017: David Lomet, Ming C. Lin, Gregory T. Byrd, Alfredo Benso, Forrest Shull, Fabrizio Lombardi, Hausi A. Müller

Term Expiring 2018: Ann DeMarle, Fred Douglass, Vladimir Getov, Bruce M. McMillin, Cecilia Metra, Kunio Uchiyama, Stefano Zanero

EXECUTIVE STAFF

Executive Director: Angela R. Burgess; **Director, Governance & Associate Executive Director:** Anne Marie Kelly; **Director, Finance & Accounting:** Sunny Hwang; **Director, Information Technology & Services:** Sumit Kacker; **Director, Membership Development:** Eric Berkowitz; **Director, Products & Services:** Evan M. Butterfield; **Director, Sales & Marketing:** Chris Jensen

COMPUTER SOCIETY OFFICES

Washington, D.C.: 2001 L St., Ste. 700, Washington, D.C. 20036-4928

Phone: +1 202 371 0101 • **Fax:** +1 202 728 9614 • **Email:** hq.ofc@computer.org

Los Alamitos: 10662 Los Vaqueros Circle, Los Alamitos, CA 90720

Phone: +1 714 821 8380 • **Email:** help@computer.org

Membership & Publication Orders

Phone: +1 800 272 6657 • **Fax:** +1 714 821 4641 • **Email:** help@computer.org

Asia/Pacific: Watanabe Building, 1-4-2 Minami-Aoyama, Minato-ku, Tokyo 107-

0062, Japan • **Phone:** +81 3 3408 3118 • **Fax:** +81 3 3408 3553 • **Email:** tokyo.ofc@computer.org

IEEE BOARD OF DIRECTORS

President & CEO: Barry L. Shoop; **President-Elect:** Karen Bartleson; **Past President:** Howard E. Michel; **Secretary:** Parviz Famouri; **Treasurer:** Jerry L. Hudgins; **Director & President, IEEE-USA:** Peter Alan Eckstein; **Director & President, Standards Association:** Bruce P. Kraemer; **Director & VP, Educational Activities:** S.K. Ramesh; **Director & VP, Membership and Geographic Activities:** Wai-Choong (Lawrence) Wong; **Director & VP, Publication Services and Products:** Sheila Hemami; **Director & VP, Technical Activities:** Jose M.F. Moura; **Director & Delegate Division V:** Harold Javid; **Director & Delegate Division VIII:** John W. Walz

revised 10 June 2016



THE ARCHITECT AS COMMUNITY SHEPHERD IN THE FUTURE

The past two decades have seen the rise of three interrelated trends that have dramatically and, we believe, irreversibly changed software development: open source, outsourcing, and distributed development. Each of these trends has changed projects' social and organizational structures. So, architects who are in charge of or interact with such projects must be aware of these trends' implications.

For nearly 50 years, developers have generally accepted Conway's law: system designs should be congruent with an organization's structure, or problems will ensue.¹ In traditional system development, in which project members were largely colocated and systems were typically small compared with current practice, Conway's law had immediate, salient consequences. Today, in the world of distributed development, misalignments between a project's social or organizational structure and its architecture might be much harder to detect and more insidious.

Consequently, architects will need to be aware of the misalignments they might face, so that they can quickly detect and correct those misalignments before they cause too much damage (for example, additional social or organizational costs²). This is a new role for architects. We've always claimed that architects are much more than just a "technical lead,"³ but now they must also be an active community *shepherd*. We chose the term "shepherd" because a shepherd guides and cares for a flock.

In much the same way that shepherds keep watch over their flocks, architects must deftly guide the social and organizational structures of today's increasingly distributed projects, looking for the early warning signs of trouble. Such signs include disengagement, institutional isomorphism, and priggishness (see Table 1 in the main article). After detecting those signs, architects must perform a series of appropriate and properly aligned mitigations (see Table 2 in the main article). So, the architect's role is changing, which implies new duties and requires upgraded skills and knowledge.

References

1. M. Conway, "How Do Committees Invent?," *Datamation*, Apr. 1968, pp. 28–31.
2. D.A. Tamburri et al., "Social Debt in Software Engineering: Insights from Industry," *J. Internet Services and Applications*, vol. 6, no. 1, 2015, pp. 10:1–10:17.
3. L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 3rd ed., 2012, Addison-Wesley.

appears to be a consequence of Conway's law: "Organizations which design systems ... are constrained to produce designs which are copies of the communication structures of these organizations."¹⁴

Consequently, the software architect's role is becoming increasingly entangled with community or "Conwayness" management duties. These duties include managing the relation between architectures and their surrounding organizational structures and protecting architectures (and projects) from suboptimal organizational and sociotechnical circumstances. Further research should concentrate on operationalizing these sociotechnical circumstances for proper quantitative appraisal and incident detection and mitigation.

Also, with modern software systems' increasing scale, speed, and sociotechnical impact, more insights into unknown community smells might be required. This could involve

- reviewing research on subversive stakeholders¹⁵ or
- investigating how well-known sociotechnical or societal phenomena such as the "tragedy of the commons"¹⁶ affect software communities and their relations to software architects and architectures.

Finally, research could look at operationalization of the community patterns reported in this article: transforming the smell patterns into automatically measurable quantities (for example, by mining data from software repositories). Using such operationalizations, we can estimate a project's social debt (caused by community smells) as a way to reason about and manage its costs. Furthermore, we intend to empirically

small-team scenarios, in which three to 10 people are bound to a time frame by contractual arrangements that establish a small, cohesive project team.

Now more than ever, software architects have to supervise the synchronization between software architectures and community structures.¹³ This

investigate the relationships between smells, architecture patterns, and community patterns.^{3,4}

For our view of where the architects' role in community shepherding is headed, see the sidebar. 🍷

References

1. L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 3rd ed., 2012, Addison-Wesley.
2. D.A. Tamburri et al., "Architecting in Networked Organizations," *Proc. 11th Working IEEE/IFIP Conf. Software Architecture (WICSA 14)*, 2014, pp. 247–256.
3. D.A. Tamburri, P. Lago, and H. van Vliet, "Organizational Social Structures for Software Engineering," *ACM Computing Surveys*, vol. 46, no. 1, 2013, article 3.
4. D.A. Tamburri, P. Lago, and H. van Vliet, "Uncovering Latent Social Communities in Software Development," *IEEE Software*, vol. 30, no. 1, 2013, pp. 29–36.
5. D.A. Tamburri and E.D. Nitto, "When Software Architecture Leads to Social Debt," *Proc. 12th Working IEEE/IFIP Conf. Software Architecture (WICSA 15)*, 2015, pp. 61–64.
6. N. Brown et al., "Managing Technical Debt in Software-Reliant Systems," *Proc. FSE/SDP Workshop Future of Software Eng. Research (FoSER 10)*, 2010, pp. 47–52.
7. D.A. Tamburri et al., "Social Debt in Software Engineering: Insights from Industry," *J. Internet Services and Applications*, vol. 6, no. 1, 2015, pp. 10:1–10:17.
8. C. McGoff, *The Primes: How Any Group Can Solve Any Problem*, John Wiley & Sons, 2012.
9. R. Kazman et al., "A Case Study in Locating the Architectural Roots of Technical Debt," *Proc. 37th Int'l Conf. Software Eng. (ICSE 15)*, 2015, pp. 179–188.

ABOUT THE AUTHORS



DAMIAN A. TAMBURRI is a research fellow at Politecnico di Milano. His research interests include social software engineering, advanced software architecture styles, and advanced software-architecting methods. He's on the *IEEE Software* editorial board and is secretary of the International Federation for Information Processing Working Group on Service-Oriented Computing. Contact him at damianandrew.tamburri@polimi.it or dtamburri@acm.org.



RICK KAZMAN is a professor of information technology management at the University of Hawaii and a principal researcher at Carnegie Mellon University's Software Engineering Institute. His research interests include software architecture design and analysis tools, software visualization, and software engineering economics. Kazman is the chair of the IEEE Technical Council on Software Engineering. Contact him at kazman@hawaii.edu.



HAMED FAHIMI is an IT project manager and consultant at CGI. His research interests include communities of practice and knowledge engineering. Fahimi received an MSc in information studies—business information systems from Universiteit van Amsterdam. Contact him at hmd.fahimi@gmail.com.

10. D. Montello, "The Measurement of Cognitive Distance: Methods and Construct Validity," *J. Environmental Psychology*, vol. 11, no. 2, 1991, pp. 101–122.
11. C. Manteli, B. van den Hooff, and H. van Vliet, "The Effect of Governance on Global Software Development: An Empirical Research in Transactive Memory Systems," *J. Information and Software Technology*, vol. 56, no. 10, 2014, pp. 1309–1321.
12. L. Bass, I. Weber, and L. Zhu, *DevOps: A Software Architect's Perspective*, Addison-Wesley, 2015.
13. R. Kazman and H.-M. Chen, "The Metropolis Model: A New Logic for the Development of Crowdsourced Systems," *Comm. ACM*, vol. 52, no. 7, 2009, pp. 76–84.
14. M. Conway, "How Do Committees Invent?," *Datamation*, Apr. 1968, pp. 28–31.
15. J. Rost and R.L. Glass, "The Impact of Subversive Stakeholders on Software Projects," *Comm. ACM*, vol. 52, no. 7, 2009, pp. 135–138.
16. G. Hardin, "The Tragedy of the Commons," *Science*, vol. 162, no. 3859, 1968, pp. 1243–47.



Selected CS articles and columns are also available for free at <http://ComputingNow.computer.org>.